

የትም

# Yetim

## Offline Field Issue Intelligence & Coordination Platform

**Document:** Software Requirements Specification (SRS)

**Version:** 1.0.0

**Date:** 1 October 2026

**Prepared for:** WEDER Strategies - Full Stack Developer Internship Technical Exercise

**Target architecture:** Next.js App Router full-stack application + PostgreSQL + IndexedDB

**Status:** Implementation Baseline / Review Draft

### Product name & brand story

የትም (Yetim) means "anywhere" in Amharic.

#### The story

- Field workers report from anywhere, including places with no signal at all.
- The app works anywhere, with or without a network.
- Reports reach the coordinator from anywhere in the field.

#### Where to use it

Tagline: "Yetim: report from anywhere."

Amharic: በከየትም ሪፖርት ኢድርጉዝ("report from anywhere").

Offline banner: "You're offline. Yetim keeps working."

README one-liner: "An offline-first field issue tracker that works anywhere, even where there's no signal."

Repository: yetim-field-issue-tracker

Confidential assessment material - prepared for technical evaluation

# Document Control

| Item                          | Value                                                                             |
|-------------------------------|-----------------------------------------------------------------------------------|
| Document ID                   | Yetim -SRS-001                                                                    |
| Version                       | 1.0.0                                                                             |
| Prepared date                 | 1 October 2026                                                                    |
| Prepared by                   | Ayalkbet Teketel                                                                  |
| Primary audience              | Technical reviewers, developers, testers, future maintainers                      |
| Assessment context            | Full Stack Developer Internship technical exercise                                |
| Source specification deadline | Friday, 2 October 2026, 9:00 AM EAT                                               |
| Primary platform              | Responsive web application / installable PWA-capable web app                      |
| Primary architecture          | Next.js App Router, TypeScript, Route Handlers, Prisma ORM, PostgreSQL, IndexedDB |
| Authentication                | Not required for assessment; role simulation supported                            |
| Deployment                    | Optional for assessment; production-ready deployment path documented              |

## Revision History

| Version | Date       | Author           | Summary                                                                                       |
|---------|------------|------------------|-----------------------------------------------------------------------------------------------|
| 0.1     | 1 Oct 2026 | Ayalkbet Teketel | Initial requirements baseline                                                                 |
| 1.0     | 1 Oct 2026 | Ayalkbet Teketel | Expanded functional, offline-sync, QA, architecture, assumptions, and acceptance requirements |

## Approval / Review Status

This document is the authoritative requirements baseline for the technical exercise implementation. Requirements marked **MUST** form the core submission scope. **SHOULD** requirements provide polish and differentiation; **MAY** requirements are optional extensions that must not compromise core correctness.

# Table of Contents

1. Executive Summary
  2. Product Vision and Goals
  3. Problem Statement and Context
  4. Users, Roles, and Operating Context
  5. Scope and Feature Map
  6. Functional Requirements
  7. Offline-First and Synchronization Requirements
  8. Workflow, State Machine, and Business Rules
  9. Data Model and Persistence Requirements
  10. Application Architecture
  11. User Experience and Interface Requirements
  12. API and Integration Requirements
  13. Validation, Error Handling, and Data Safety
  14. Security, Privacy, and Reliability
  15. Testing and Quality Assurance
  16. Deployment and Environment Requirements
  17. Observability and Auditability
  18. Assumptions, Ambiguities, and Decisions
  19. Acceptance Criteria and Traceability
  20. Implementation Priorities and Delivery Plan
  21. Known Limitations and Future Enhancements
- Appendix A.** Demo / Manual QA Checklist
- Appendix B.** Suggested Repository Structure
- Appendix C.** AI and Development-Tool Disclosure Plan
- Appendix D.** References

## Navigation note

The final implementation should preserve this section numbering in the README so reviewers can map requirements to code, tests, and acceptance checks.

# 1. Executive Summary

Yetim is an offline-first field issue reporting and coordination platform for workers who may have intermittent or unavailable internet connectivity. It captures infrastructure problems such as broken water points, damaged equipment, service interruptions, safety concerns, and maintenance needs; preserves the report locally; synchronizes safely when connectivity returns; and provides coordinators with a clear operational workflow from submission to resolution.

The system is intentionally designed as a full-stack Next.js application so the frontend and backend can share one repository, deployment pipeline, validation layer, types, and domain model. Next.js positions itself as a React framework for full-stack web applications, and its App Router supports server-side application structure and Route Handlers for API endpoints. [Ref: Next.js Documentation; Next.js Learn - Fetching Data]

The defining engineering challenge is not ordinary CRUD. The platform must protect the user from data loss while offline, make synchronization visible, prevent duplicate server records, preserve history, and behave predictably when network conditions or data versions change.

## Success definition

A reviewer should be able to create a report while offline, refresh or reopen the application without losing it, see a clear pending state, restore connectivity, synchronize without duplicates, inspect the resulting server record, move it through valid workflow states, observe the history, and see clean handling of failures and invalid actions.

## 2. Product Vision and Goals

### 2.1 Vision

Provide field teams with a trustworthy “report once, never lose it” operational tool that turns local observations into actionable, auditable infrastructure work items even when connectivity is unreliable.

### 2.2 Product goals

- Capture high-quality issue reports quickly from mobile or desktop browsers.
- Continue working during complete network outages without silently discarding user input.
- Synchronize pending work safely, transparently, and idempotently.
- Give coordinators one operational view of what needs attention, what is in progress, and what is resolved.
- Preserve an auditable history of important report and synchronization events.
- Provide enough analytics and filtering to help coordinators prioritize without turning the exercise into a full enterprise ERP.
- Keep the architecture understandable, testable, deployable, and maintainable within a single Next.js codebase.

### 2.3 Design principles

- Offline-first, not offline-afterthought.
- Explicit state over hidden magic.
- Idempotent writes over duplicate-prone retries.
- Server validation over trust in the browser.
- Human-readable failures over silent retries.
- Progressive enhancement for unreliable networks and devices.
- Simple core domain with extensible boundaries.

### 3. Problem Statement and Context

Field workers may be inspecting roads, water infrastructure, equipment, facilities, utility points, or service locations where cellular data is weak or unavailable. A conventional online-only form creates operational risk: a report may fail to submit, a worker may not know whether the report exists, the user may duplicate the report after retrying, or the local browser may lose the data after a refresh.

#### 3.1 Specification-derived problem scenarios

| Scenario                                         | Risk                               | response                                                          |
|--------------------------------------------------|------------------------------------|-------------------------------------------------------------------|
| Broken water point found with no connectivity    | Worker cannot submit now           | Save locally and mark pending                                     |
| Worker refreshes after creating report           | Data may disappear                 | Persist report and queue in IndexedDB                             |
| Connection returns after several reports         | Repeated manual entry / duplicates | Queue-based synchronization with client-generated idempotency key |
| Coordinator changes status while device is stale | Blind overwrite / inconsistency    | Version-aware conflict detection and explicit resolution          |
| Network drops during synchronization             | Partial data ambiguity             | Per-operation result state, retryable queue, history event        |
| Invalid workflow action                          | Inconsistent status                | Server-enforced transition matrix and clear error                 |

#### 3.2 Non-goals for the assessment baseline

- Enterprise identity management and SSO.
- Complex multi-tenant billing or subscription management.
- Real-time collaboration comparable to a messaging system.
- Machine-learning diagnosis of infrastructure faults.
- Guaranteed background synchronization on every browser; background sync is treated as an enhancement because browser support and execution conditions vary.

## 4. Users, Roles, and Operating Context

| Role            | Primary responsibilities                 | Key capabilities                                                                                                          |
|-----------------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Field Worker    | Observe and report issues from the field | Create draft/report, edit drafts, view own/pending records, trigger sync, inspect sync errors, optionally attach evidence |
| Coordinator     | Review and operationalize reports        | View/filter all reports, assign, prioritize, change status, reject, resolve, inspect history, resolve conflicts           |
| Demo / Reviewer | Evaluate behavior and technical quality  | Switch simulated role, toggle offline mode, inspect sync queue, replay failure scenarios, run test dataset                |

### 4.1 Operating environments

- Mobile-first field use on modern mobile browsers.
- Desktop coordinator use on modern Chromium/Firefox/Safari-class browsers.
- Intermittent connectivity, long offline periods, captive portals, high latency, temporary API failures.
- Browser storage is available but may be quota-limited; attachments therefore require explicit size controls.
- The demo supports a simulation control for offline mode so reviewers can deterministically evaluate the required behavior without physically disabling network access.

### 4.2 Role simulation

Authentication is not required by the exercise. Yetim should nevertheless model the distinction between field-worker and coordinator permissions in the domain layer. A role switcher may be provided in development/demo mode; production deployment can replace it with real identity and authorization without changing the core report domain.

## 5. Scope and Feature Map

| Capability               | Priority | Included behavior                                                                                                             |
|--------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------|
| Report creation          | MUST     | Category, description, location, priority, status, reported date/time, local client ID, optional reporter and evidence fields |
| Offline persistence      | MUST     | IndexedDB persistence across refresh/reopen                                                                                   |
| Sync queue               | MUST     | Pending/Syncing/Synchronized/Failed/Conflict states; retry; idempotency                                                       |
| Status workflow          | MUST     | Draft -> Submitted -> Assigned -> In Progress -> Resolved; Rejected path; validation of transitions                           |
| Reports list and details | MUST     | Filtering, sorting, search, sync state, priority, status, history                                                             |
| History/audit trail      | MUST     | Creation, sync, status changes, failures, assignment/priority changes                                                         |
| Validation/error states  | MUST     | Client + server validation, visible error feedback                                                                            |
| Automated tests          | MUST     | API/domain/offline/sync critical behavior                                                                                     |
| Coordinator workbench    | SHOULD   | Kanban-style board, quick filters, assignment and priority editing                                                            |
| Map/location enrichment  | SHOULD   | Optional coordinates, accuracy, map preview where browser permission is available                                             |
| Photo attachments        | SHOULD   | Optional compressed evidence with local pending state                                                                         |
| Dashboard analytics      | SHOULD   | Counts, trends, SLA-style aging indicators                                                                                    |
| Export                   | MAY      | CSV/JSON export for reviewer/demo utility                                                                                     |
| Notifications            | MAY      | In-app notification center; real push notifications are outside baseline                                                      |
| Internationalization     | MAY      | English-first, Amharic-ready strings architecture                                                                             |

### Scope rule

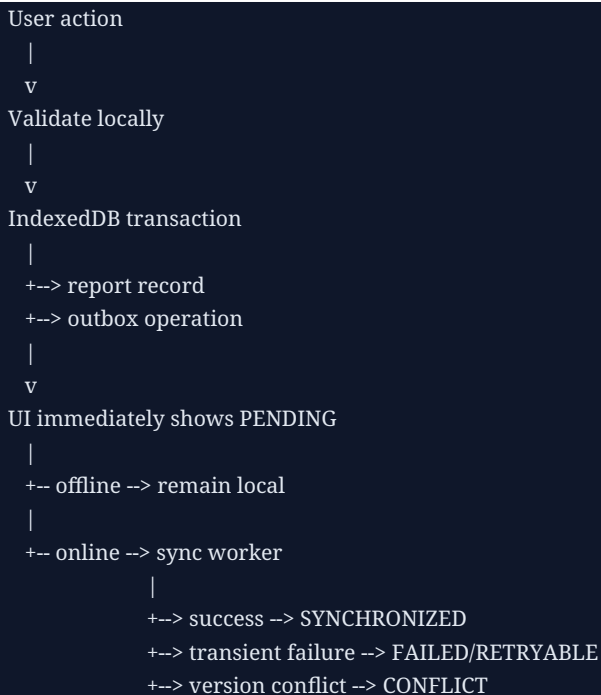
Every MUST item is required for the submission. SHOULD features are permitted only when they do not weaken synchronization correctness, tests, README quality, or source-control discipline.

## 6. Functional Requirements

| ID    | Area                   | Priority | Requirement                                                                                                                         |
|-------|------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | Report creation        | MUST     | Create a report with category, description, location, priority, status, reported date/time, and a generated clientId.               |
| FR-02 | Draft support          | MUST     | Allow a partially completed report to be saved as Draft locally; drafts are not submitted to the server until explicitly submitted. |
| FR-03 | Validation             | MUST     | Reject invalid values with field-level feedback before local persistence and again at the server boundary.                          |
| FR-04 | Offline creation       | MUST     | Allow a valid report to be saved while offline and immediately show its pending synchronization state.                              |
| FR-05 | Persistence            | MUST     | Persist reports and synchronization queue records in IndexedDB so refresh/reopen does not erase them.                               |
| FR-06 | Reports list           | MUST     | Display local and synchronized reports with status, priority, reported time, and synchronization state.                             |
| FR-07 | Report detail          | MUST     | Show complete report fields, current workflow status, sync state, identifiers, and history.                                         |
| FR-08 | Manual sync            | MUST     | Provide a visible Sync Now action that processes retryable pending work.                                                            |
| FR-09 | Automatic sync trigger | MUST     | Attempt synchronization when the app detects restored connectivity and on selected lifecycle events such as focus/resume.           |
| FR-10 | Sync result            | MUST     | Represent each pending operation as synchronized, failed, or conflict; failed work remains retryable.                               |
| FR-11 | Duplicate prevention   | MUST     | Use client-generated UUID/idempotency key and server uniqueness constraint to prevent duplicate records on retry.                   |
| FR-12 | Workflow               | MUST     | Enforce the defined report status transition matrix on the server.                                                                  |
| FR-13 | History                | MUST     | Record creation, successful synchronization, synchronization failure, status changes, and relevant edits.                           |
| FR-14 | Coordinator review     | MUST     | Coordinator role can view the complete operational report set and update workflow state.                                            |
| FR-15 | Filtering              | MUST     | Filter by status, priority, category, synchronization state, and date range.                                                        |
| FR-16 | Search                 | SHOULD   | Search by issue ID, client ID, description, and location.                                                                           |
| FR-17 | Assignment             | SHOULD   | Coordinator can assign a report to a simulated field worker/team member.                                                            |
| FR-18 | Priority change        | SHOULD   | Coordinator can change priority with a history event.                                                                               |
| FR-19 | Evidence attachment    | SHOULD   | Allow optional image/file evidence with controlled size/type and local persistence state.                                           |
| FR-20 | Location enrichment    | SHOULD   | Capture optional latitude/longitude and accuracy after explicit user permission.                                                    |
| FR-21 | Conflict resolution    | SHOULD   | Surface version conflicts rather than silently overwriting server state; provide explicit resolution options.                       |
| FR-22 | Analytics              | SHOULD   | Show operational summary metrics without changing source-of-truth report data.                                                      |

## 7. Offline-First and Synchronization Requirements

### 7.1 Local-first record lifecycle



### 7.2 Connectivity model

Browser connectivity indicators alone are not treated as proof that the backend is reachable. The UI may use `navigator.onLine` as an early hint, but a successful lightweight API health/readiness probe should establish application-level connectivity before a sync attempt is considered online.

### 7.3 Queue semantics

- Each local mutation that requires server delivery is represented in an outbox queue.
- Queue records contain operation type, report `clientId`, local operation ID, attempt count, timestamps, last error, and optional base version.
- Operations are processed deterministically; a failing item does not delete later items.
- Successful operations are acknowledged and removed or marked complete in the local outbox.
- Transient failures remain retryable with bounded exponential backoff plus manual retry.
- Permanent validation failures are surfaced to the user and require correction rather than endless retries.

### 7.4 Idempotency / duplicate prevention

Every report generated locally receives a UUID `clientId` before any server request. The server stores this identifier under a unique constraint. Replaying the same CREATE operation therefore returns the existing report rather than creating a second record. The client treats an idempotent “already exists / same operation acknowledged” response as synchronization success.

### 7.5 Interrupted synchronization

Synchronization is transactional at the operation level, not falsely presented as all-or-nothing across the entire queue. If a network failure occurs after some reports were accepted, successfully acknowledged operations remain synchronized while the remaining operations remain pending or failed. The UI displays the exact result per report.

### 7.6 Retry policy

| Failure class | Example                         | Client behavior                                    |
|---------------|---------------------------------|----------------------------------------------------|
| Transient     | Timeout, 502/503, network reset | Keep queued; bounded retry/backoff; manual retry   |
| Validation    | Missing field, invalid enum     | Mark failed; correct input; retry after correction |

| Failure class | Example          | Client behavior                                                        |
|---------------|------------------|------------------------------------------------------------------------|
| Conflict      | Version mismatch | Mark conflict; show server/local values; require explicit resolution   |
| Authorization | Role violation   | Do not retry blindly; show permission error                            |
| Unknown       | Unexpected error | Keep operation safe; capture diagnostic context; cap automatic retries |

## 7.7 Synchronization UX contract

| State        | Visible meaning                                | Available action                                         |
|--------------|------------------------------------------------|----------------------------------------------------------|
| Pending      | Saved locally; not yet acknowledged by server  | Sync Now                                                 |
| Syncing      | <b>Delivery is in progress</b>                 | Show progress; avoid duplicate manual sends              |
| Synchronized | Server acknowledgement received                | View server ID; continue normal workflow                 |
| Failed       | Last attempt did not complete                  | View reason; Retry after transient failure or correction |
| Conflict     | Local operation targets a stale server version | Review difference; explicitly resolve                    |

## 7.8 Synchronization consistency invariants

- A pending local report is never deleted merely because a request failed.
- A successful acknowledgement is recorded before the local outbox item is considered complete.
- Retries are safe because create operations are keyed by immutable clientId.
- A conflict never silently chooses local or server state without an explicit resolution rule.
- The user can always see whether the current copy is local-only, synchronized, failed, or conflicted.

### Data safety invariant

At no point may a sync attempt delete the only known copy of a report before server acknowledgement. Local data must survive failed requests and application restarts.

## 8. Workflow, State Machine, and Business Rules

### 8.1 Status state machine

```
DRAFT
| submit
v
SUBMITTED
| assign
v
ASSIGNED
| start work
v
IN_PROGRESS
| complete
v
RESOLVED

SUBMITTED --reject--> REJECTED

Optional controlled reopening:
RESOLVED --reopen--> IN_PROGRESS
```

### 8.2 Valid transition matrix

| From        | To          | Trigger / rule                         |
|-------------|-------------|----------------------------------------|
| DRAFT       | SUBMITTED   | Submit report                          |
| SUBMITTED   | ASSIGNED    | Coordinator assigns work               |
| SUBMITTED   | REJECTED    | Coordinator rejects                    |
| ASSIGNED    | IN_PROGRESS | Work begins                            |
| IN_PROGRESS | RESOLVED    | Work completed                         |
| RESOLVED    | IN_PROGRESS | Optional explicit reopen               |
| REJECTED    | SUBMITTED   | Optional resubmission after correction |

### 8.3 Business rules

- FR-BR-01: Every report must have an immutable clientId.
- FR-BR-02: Draft records are editable by the creator in the local application.
- FR-BR-03: Submitted records should not allow uncontrolled edits to core evidence fields; coordinator changes are explicit and audited.
- FR-BR-04: Status transitions are server-validated even if the UI already hides invalid buttons.
- FR-BR-05: Resolved and Rejected are terminal from the normal worker flow unless an explicit reopen/resubmit transition is supported.
- FR-BR-06: Changing status, priority, assignment, or important report fields creates a history record.
- FR-BR-07: ReportedAt reflects the device event time; serverReceivedAt records when the backend actually received it.
- FR-BR-08: The system must never infer that a report was synchronized solely because the UI sent a request; acknowledgement is required.

## 9. Data Model and Persistence Requirements

### 9.1 Core entities

| Entity          | Key fields                                                                                             | Purpose                           |
|-----------------|--------------------------------------------------------------------------------------------------------|-----------------------------------|
| Report          | id, clientId, category, description, location, priority, status, reportedAt, serverReceivedAt, version | Authoritative issue record        |
| OutboxOperation | id, clientOperationId, reportClientId, operation, payload, status, attempts, lastError, baseVersion    | Reliable local delivery queue     |
| ReportHistory   | id, reportId, action, actorRole, fromValue, toValue, message, createdAt                                | Audit trail                       |
| Assignment      | reportId, assigneeId, assigneeName                                                                     | Optional ownership model          |
| Attachment      | id, reportId/clientId, type, size, localUri, remoteUri, checksum, status                               | Optional evidence files           |
| SyncSession     | id, startedAt, completedAt, successCount, failureCount                                                 | Optional operational sync summary |

### 9.2 Local vs server storage

| Data                       | Local IndexedDB                                   | Server PostgreSQL               |
|----------------------------|---------------------------------------------------|---------------------------------|
| Drafts                     | Yes                                               | No until submitted              |
| Pending submitted reports  | Yes                                               | Yes after successful sync       |
| Outbox operations          | Yes                                               | No                              |
| Authoritative report state | Cached copy                                       | Yes                             |
| History                    | Cached copy                                       | Yes                             |
| Large attachments          | Prefer compressed blob / object storage reference | Yes/reference, subject to scope |
| Sync metadata              | Yes                                               | Optional server audit metadata  |

### 9.3 Versioning

Server-side report records should include an integer version or equivalent updatedAt-based concurrency token. A client mutation may include its baseVersion. If the current server version differs, the server returns a conflict rather than silently overwriting the record.

### 9.4 Time model

Persist timestamps in UTC at the server. Preserve the originating device timestamp and timezone offset when useful for auditability. The UI renders time in the user's local timezone while allowing the raw timestamp to be inspected in details/history.

## 10. Application Architecture

### 10.1 Chosen architecture

Yetim will use one Next.js App Router repository with a clear separation between presentation, domain logic, persistence, and offline infrastructure. Next.js supports full-stack application development and Route Handlers for API endpoints, making a single deployment unit practical for this exercise. [Ref: Next.js Documentation; Next.js Learn - Fetching Data]

```
Browser / PWA layer
  App Router pages + client components
  Offline service + IndexedDB repository
  Sync engine + outbox
  UI state + network state
    |
    v
Next.js server layer
  Route Handlers /api/*
  Validation + authorization boundary
  Domain services / state machine
    |
    v
Persistence
  Prisma
    |
    v
PostgreSQL
```

### 10.2 Proposed layers

- UI layer: pages, layouts, reusable components, forms, dashboards, accessibility states.
- Application layer: use cases such as createReport, submitReport, syncPendingOperations, changeStatus.
- Domain layer: status transition rules, validation schemas, idempotency rules, conflict semantics.
- Data layer: Prisma repositories and IndexedDB repository adapters.
- Infrastructure layer: network probes, service worker registration, sync scheduling, logging.

### 10.3 Why one Next.js application

- Single deployment target and simpler review.
- Shared TypeScript models and validation contracts reduce drift.
- Server code remains isolated from browser bundles.
- App Router gives clear route structure and nested layouts.
- The architecture can later split the API if scale requires it without redesigning the core domain.

## 11. User Experience and Interface Requirements

### 11.1 UX principles

- Offline state must always be visible.
- Primary field actions should require minimal taps/clicks.
- The interface should feel trustworthy and calm during synchronization.
- Errors should explain what happened and what the user can do next.
- Color must not be the only signal for status or priority.
- Mobile touch targets and responsive tables/cards are required.
- The coordinator experience should favor scanability: filters, counts, status groupings, and clear timestamps.

### 11.2 Required screens

| Screen                 | Required elements                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------|
| Landing / workspace    | Product identity, current network state, role, quick create, operational summary                    |
| Create / Edit Report   | Validated form, draft save, required fields, optional location/evidence, clear offline messaging    |
| Reports                | Search, filter, sort, tabs/chips for status and sync state, list/card view                          |
| Report Details         | All fields, status control, sync status, history timeline, assignment/priority actions by role      |
| Sync Center            | Pending queue count, last sync, success/failure summaries, retry controls, expandable error details |
| Coordinator Board      | Columns for workflow states; quick filters; assignment and priority actions                         |
| Analytics              | Counts by status/category/priority, aging overview, sync reliability indicators                     |
| Settings / Diagnostics | Demo role, simulated offline toggle, local data management, application version                     |

### 11.3 Trust indicators

| Indicator    | Meaning                                                         |
|--------------|-----------------------------------------------------------------|
| OFFLINE      | Browser or app cannot currently reach the backend               |
| PENDING      | Local copy exists and waits for synchronization                 |
| SYNCING      | Operation is actively being delivered                           |
| SYNCHRONIZED | Server acknowledgement received                                 |
| FAILED       | Operation did not complete and is retryable or needs correction |
| CONFLICT     | Server and local versions diverged; explicit resolution needed  |

## 12. API and Integration Requirements

### 12.1 Endpoint baseline

| Method | Endpoint                 | Purpose                                 |
|--------|--------------------------|-----------------------------------------|
| GET    | /api/health              | Basic process health                    |
| GET    | /api/ready               | Dependency/readiness check              |
| GET    | /api/reports             | List/filter reports                     |
| POST   | /api/reports             | Create/sync report idempotently         |
| GET    | /api/reports/:id         | Fetch report details                    |
| PATCH  | /api/reports/:id         | Controlled field updates                |
| POST   | /api/reports/:id/status  | Execute validated status transition     |
| GET    | /api/reports/:id/history | Fetch audit history                     |
| POST   | /api/sync                | Optional batch synchronization endpoint |
| GET    | /api/sync/status         | Summary of sync/reliability state       |

### 12.2 API contract rules

- All mutable input is validated on the server.
- Use consistent JSON success/error envelopes.
- Use 409 Conflict for version/idempotency conflicts where appropriate.
- Use 422 or 400 for validation errors.
- Use 5xx only for unexpected server failures.
- Never expose database credentials or stack traces to the browser.
- Responses should include stable error codes suitable for UI mapping.

### 12.3 Example error envelope

```
{
  "error": {
    "code": "INVALID_STATUS_TRANSITION",
    "message": "Cannot move a DRAFT report directly to RESOLVED.",
    "details": {"from": "DRAFT", "to": "RESOLVED"}
  }
}
```

## 13. Validation, Error Handling, and Data Safety

### 13.1 Validation layers

| Layer              | Responsibility                                                              |
|--------------------|-----------------------------------------------------------------------------|
| UI                 | Fast feedback, field constraints, required/format checks                    |
| Application/domain | Shared schemas and business rules                                           |
| API boundary       | Treat browser input as untrusted; validate again                            |
| Database           | Unique constraints, foreign keys, enum/domain constraints where appropriate |

### 13.2 Failure UX

- Every actionable failure has a stable code and human-readable explanation.
- Failed synchronization shows the last failure reason and timestamp.
- Users can retry without re-entering a report.
- Permanent validation failures point to the exact corrective field.
- The UI does not silently convert failures into successful states.

### 13.3 Local data protection rules

- Write report and corresponding outbox record atomically where the IndexedDB transaction model permits.
- Do not clear local records after an arbitrary logout/session reset in the assessment build.
- Provide a deliberate “Clear local data” action behind an explicit confirmation.
- Keep attachment size bounded and compress where practical.
- Never place secrets or database credentials in IndexedDB.

# 14. Security, Privacy, and Reliability

## 14.1 Security baseline

- Server-side authorization checks despite simulated roles.
- Strict input validation to reduce malformed or malicious data.
- Parameterized ORM/database access through Prisma.
- Do not store secrets in source control.
- Use HTTPS in deployed environments.
- Use secure HTTP headers appropriate for the chosen deployment.
- Limit file types and sizes for attachments.
- Avoid exposing internal stack traces in production responses.

## 14.2 Privacy baseline

The assessment does not require authentication and should avoid collecting unnecessary personal information. Optional reporter names should be aliases or demonstration values. Location data is sensitive in many field contexts; Yetim should capture GPS only after explicit permission and make it visible that location enrichment is optional.

## 14.3 Reliability objectives

| Objective              | Target behavior                                                             |
|------------------------|-----------------------------------------------------------------------------|
| No silent data loss    | A locally saved report remains recoverable until explicit user deletion.    |
| No duplicate creates   | Repeated synchronization of the same clientId results in one server report. |
| Failure transparency   | Every sync failure is represented and inspectable.                          |
| Restart resilience     | Pending queue survives browser refresh/reopen.                              |
| Deterministic workflow | Invalid transitions fail predictably and are auditable.                     |

## 15. Testing and Quality Assurance

### 15.1 Test strategy

The test suite should prioritize behaviors that are easy to get subtly wrong: persistence, idempotent synchronization, workflow transitions, conflict handling, and history generation. UI snapshot volume is less valuable than a few high-signal behavioral tests.

### 15.2 Required automated tests

| ID   | Layer   | Test                                                                 |
|------|---------|----------------------------------------------------------------------|
| T-01 | Domain  | Valid status transition accepted                                     |
| T-02 | Domain  | Invalid status transition rejected                                   |
| T-03 | API     | Invalid report payload rejected                                      |
| T-04 | API     | Duplicate clientId returns existing logical record                   |
| T-05 | API     | Status change creates history entry                                  |
| T-06 | Offline | Report survives refresh/reopen in IndexedDB                          |
| T-07 | Sync    | Pending report becomes synchronized after successful acknowledgement |
| T-08 | Sync    | Failed request remains in retryable queue                            |
| T-09 | Sync    | Retry does not create duplicate server record                        |
| T-10 | Sync    | Version conflict is surfaced and not silently overwritten            |
| T-11 | History | Create + sync + status changes produce expected chronological events |
| T-12 | UI      | Offline banner / sync-state badges render correct state              |

### 15.3 Manual QA scenarios

- Create report online and verify immediate server appearance.
- Switch to simulated offline, create report, refresh, and confirm persistence.
- Create multiple offline reports, restore connectivity, sync, and confirm all appear once.
- Force a network failure during sync and verify failed/pending behavior.
- Repeat sync and verify no duplicates.
- Attempt invalid status transition through UI and API.
- Change status through full workflow and inspect history.
- Create stale local version and provoke a server-side update to verify conflict behavior.
- Test mobile viewport, empty states, long descriptions, slow network, and validation errors.

### 15.4 Definition of done for a feature

- Requirement implemented
- Happy path tested
- Failure path considered
- State persisted correctly
- User feedback visible
- README updated if architecture/behavior changed
- Meaningful Git commit created

## 16. Deployment and Environment Requirements

### 16.1 Deployment target

The architecture is designed for a single Next.js deployment with a managed PostgreSQL-compatible database. Vercel or a comparable platform can host the Next.js application; the SRS does not require a specific vendor.

### 16.2 Environment variables

```
DATABASE_URL=...
NEXT_PUBLIC_APP_URL=...
# Optional: storage credentials if attachments are enabled
# Optional: AUTH_* variables when real authentication is introduced
```

### 16.3 Environment separation

- Development database / local Postgres
- Test database or isolated test schema
- Production deployment environment when deployed
- Never commit .env files containing secrets

### 16.4 Operational readiness

The application should expose /api/health for process-level health and /api/ready for dependency readiness. A deployment is considered usable only when the application can start, connect to its database, run migrations/seed logic as appropriate, and answer its readiness check.

## 17. Observability and Auditability

### 17.1 What should be observable

- API request failures and response classes
- Synchronization attempts, successes, failures, and conflict counts
- Report status transitions
- Queue size and oldest pending item age
- Application version and environment
- Database connectivity/readiness

### 17.2 User-facing diagnostics

A small Sync Center should expose “last successful sync,” pending count, failed count, and the most recent failure reason. This is intentionally operational rather than technical: a field worker does not need a stack trace, but a reviewer should be able to see what happened.

### 17.3 Audit requirements

History records are immutable append-only records. If a record is corrected, the correction becomes a new event rather than rewriting the audit trail.

## 18. Assumptions, Ambiguities, and Decisions

The source specification intentionally leaves several behaviors unspecified. The following decisions make those ambiguities explicit so implementation and evaluation remain deterministic.

| ID   | Ambiguity / hidden assumption                          | Decision                                                                                                                                                   |
|------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A-01 | What counts as online?                                 | A browser online flag is only a hint; successful application-level readiness probing is the stronger signal.                                               |
| A-02 | Can the worker lose data after refresh?                | No. Submitted/pending local records and drafts persist in IndexedDB until explicit deletion.                                                               |
| A-03 | How are duplicates identified?                         | By client-generated immutable UUID/clientId, not fuzzy content matching. Two genuinely separate issues may have similar descriptions.                      |
| A-04 | What if sync is interrupted halfway?                   | Per-operation acknowledgement. Completed operations stay completed; uncompleted operations remain queued.                                                  |
| A-05 | What if server state changed while device was offline? | Use version-aware mutation and explicit conflict state; never silently overwrite newer server data.                                                        |
| A-06 | What is the source of reported time?                   | The client event time is preserved; the server also stores received time. UTC is the canonical persistence format.                                         |
| A-07 | Who can change status?                                 | Field workers can perform worker-appropriate transitions; coordinators can perform operational transitions. Server enforces role rules.                    |
| A-08 | Can a resolved issue reopen?                           | Reopening is an explicit controlled action. The baseline may allow RESOLVED -> IN_PROGRESS only via coordinator action.                                    |
| A-09 | Can submitted fields be edited?                        | Core issue evidence is effectively immutable after submission; controlled corrections generate history. Drafts remain editable.                            |
| A-10 | Should offline data be encrypted?                      | IndexedDB protection depends on the browser/device security model; sensitive environments may require device management/encryption beyond this assessment. |
| A-11 | Should attachments synchronize?                        | Yes when the feature is enabled; attachment upload is treated as a separate idempotent operation and does not block the text report from being             |
| A-12 | Should background sync be guaranteed?                  | No. Use online/focus events and explicit Sync Now; optional background execution is opportunistic.                                                         |
| A-13 | What if browser storage is full?                       | The save operation must fail visibly and recommend removing old local data or attachments; it must not falsely report success.                             |
| A-14 | What if the same human submits twice intentionally?    | The system does not infer duplicates from content; the user/reviewer can flag duplicates later as an operational action.                                   |
| A-15 | What is the authority for server state?                | The server is authoritative after synchronization, while the local store is authoritative for the worker's unsynchronized work.                            |

### Why these assumptions matter

These decisions remove the most dangerous “implementation may vary” gaps: connectivity detection, duplicate prevention, retry behavior, version conflicts, time semantics, permissions, and local storage failure. Reviewers can therefore test the system against explicit, reproducible rules.

## 19. Acceptance Criteria and Traceability

### 19.1 End-to-end acceptance scenarios

| ID    | Area               | Acceptance criterion                                                                                                             |
|-------|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| AC-01 | Offline capture    | With network unavailable, user creates valid report; report appears immediately as Pending and remains after refresh/reopen.     |
| AC-02 | Reconnect + sync   | When backend becomes reachable, pending report synchronizes and displays Synchronized plus server identifier.                    |
| AC-03 | No duplicates      | Running synchronization repeatedly produces exactly one server report for a given clientId.                                      |
| AC-04 | Partial failure    | When one operation fails, successful operations remain synchronized and failed operation remains retryable.                      |
| AC-05 | Invalid transition | Attempting an undefined status transition returns a clear error and leaves the original status unchanged.                        |
| AC-06 | Full workflow      | A valid report can travel through Draft -> Submitted -> Assigned -> In Progress -> Resolved and history records each transition. |
| AC-07 | Rejected flow      | A Submitted report can be Rejected and the event is visible in history.                                                          |
| AC-08 | Coordinator view   | Coordinator can filter and inspect pending/submitted reports and update workflow status.                                         |
| AC-09 | History            | Create, sync, failure, and status changes are chronologically visible for a report.                                              |
| AC-10 | Validation         | Invalid category/description/location/priority data cannot be accepted by the backend.                                           |

### 19.2 Specification traceability

| Specification item | Requirement        | Design mapping             | Verification             |
|--------------------|--------------------|----------------------------|--------------------------|
| Spec 1             | Create reports     | FR-01..FR-03               | T-03, AC-10              |
| Spec 2             | Work offline       | FR-04..FR-07               | T-06, AC-01              |
| Spec 3             | Synchronize        | FR-08..FR-11, Section 7    | T-07..T-10, AC-02..AC-04 |
| Spec 4             | Status workflow    | FR-12, Section 8           | T-01,T-02, AC-05..AC-07  |
| Spec 5             | View reports       | FR-06..FR-07, FR-14..FR-15 | T-12, AC-08              |
| Spec 6             | History            | FR-13                      | T-05,T-11, AC-09         |
| Spec 7             | Testing            | Section 15                 | Test suite + manual QA   |
| Technical          | Frontend + backend | Section 10 + Section 12    | Build/run + API tests    |
| Technical          | Persistent storage | Section 9                  | Persistence tests        |
| Technical          | README / decisions | Section 18 + Appendix A-D  | Repository documentation |

## 20. Implementation Priorities and Delivery Plan

The application may be made visually rich and featureful, but engineering time must be protected around the evaluation-critical path. The recommended implementation order below keeps the strongest features first.

| Priority | Workstream              | Completion condition                                                         |
|----------|-------------------------|------------------------------------------------------------------------------|
| P0       | Project + DB + domain   | Next.js app runs; schema/migrations work; validation and state machine exist |
| P0       | Report CRUD             | Create/list/detail/update flows work                                         |
| P0       | IndexedDB offline layer | Create/persist/restore reports and outbox                                    |
| P0       | Sync engine             | Online detection, queue, idempotency, retries, visible states                |
| P0       | Workflow + history      | Valid transitions + immutable history                                        |
| P0       | Tests                   | Critical unit/API/offline/sync tests pass                                    |
| P0       | README + Git            | Architecture, assumptions, QA, AI disclosure, meaningful commits             |
| P1       | Coordinator polish      | Board, filters, assignment, analytics                                        |
| P1       | Evidence/location       | Image compression, coordinates, map preview                                  |
| P2       | Optional extras         | Export, notifications, advanced reporting                                    |

### 20.1 Definition of submission readiness

- Fresh clone installs and runs from README instructions.
- Test command works and critical tests pass.
- Offline scenario can be demonstrated deterministically.
- No obvious console errors or broken navigation in primary flows.
- No secrets or dependency folders committed.
- Git history reflects actual incremental development.
- Repository is public and accessible without a permission request.
- README contains all exercise-requested documentation sections.

## 21. Known Limitations and Future Enhancements

| Limitation in assessment build                        | Future enhancement                                                       |
|-------------------------------------------------------|--------------------------------------------------------------------------|
| Simulated roles; no production identity               | OAuth/SSO, RBAC/ABAC, organization/region scoping                        |
| Browser storage only for offline cache                | Native device storage / encrypted local database for managed deployments |
| Optional attachments may use simple object storage    | Resumable uploads, thumbnails, offline media gallery                     |
| Conflict flow is explicit rather than fully automatic | Field-level merge strategies and collaborative operational model         |
| Analytics are operational summaries                   | Trend forecasting, SLA dashboards, geospatial heatmaps                   |
| Background sync is opportunistic                      | Native background sync where platform support allows                     |
| English-first interface                               | Full Amharic and multilingual localization                               |

### 21.1 Product evolution

The architecture leaves room for a broader field operations platform: scheduled inspections, assets, maintenance plans, service-level targets, richer geospatial views, offline reference materials, and integrations with organizational systems. These are deliberately separated from the core issue-reporting domain so that they do not complicate the assessment build.

## Appendix A - Demo / Manual QA Checklist

- [ ] 01 Application launches successfully from a clean checkout.
- [ ] 02 Create a valid report while online.
- [ ] 03 Open report details and verify identifiers and timestamps.
- [ ] 04 Switch to simulated offline mode.
- [ ] 05 Create a report with category, description, location, priority.
- [ ] 06 Confirm Pending synchronization badge appears.
- [ ] 07 Refresh browser while offline and confirm report persists.
- [ ] 08 Create at least three pending reports.
- [ ] 09 Restore online state.
- [ ] 10 Run Sync Now and confirm successful reports show Synchronized.
- [ ] 11 Re-run Sync Now and confirm no duplicates appear.
- [ ] 12 Force a failed synchronization request and verify Failed state.
- [ ] 13 Retry the failed item and confirm recovery.
- [ ] 14 Perform valid status transitions through the workflow.
- [ ] 15 Attempt invalid status transition and confirm rejection.
- [ ] 16 Inspect history for creation, sync, status change, and failure.
- [ ] 17 Use coordinator role to filter and review all reports.
- [ ] 18 Test empty states, long descriptions, invalid fields, and slow-network behavior.
- [ ] 19 Run automated test suite and capture result in README.
- [ ] 20 Confirm .env / secrets are excluded from Git.
- [ ] 21 Confirm public repository access works in an incognito browser.

## Appendix B - Suggested Repository Structure

```
yetim-field-issue-tracker/  
├─ app/  
│ └─ (workspace)/  
│   │ └─ dashboard/  
│   │ └─ reports/  
│   │ └─ reports/[id]/  
│   │ └─ sync/  
│   └─ settings/  
├─ api/  
│   │ └─ health/  
│   │ └─ ready/  
│   │ └─ reports/  
│   └─ sync/  
└─ layout.tsx  
├─ components/  
│   │ └─ reports/  
│   │ └─ dashboard/  
│   │ └─ sync/  
│   └─ ui/  
├─ lib/  
│   │ └─ domain/  
│   │   │ └─ reports/  
│   │   │ └─ workflow/  
│   │   └─ sync/  
│   │ └─ db/  
│   │ └─ offline/  
│   │   │ └─ indexeddb.ts  
│   │   └─ sync-engine.ts  
│   │ └─ validation/  
│   └─ diagnostics/  
├─ prisma/  
│   │ └─ schema.prisma  
│   └─ seed.ts  
├─ tests/  
│   │ └─ unit/  
│   │ └─ api/  
│   └─ e2e/  
├─ public/  
├─ README.md  
├─ .env.example  
└─ package.json
```

This structure is a recommendation, not a rigid directory contract. The core expectation is clear boundaries: UI, domain logic, offline persistence/sync, server persistence, and tests should be easy to locate.

## Appendix C - AI and Development-Tool Disclosure Plan

The technical exercise explicitly permits AI-assisted development and asks candidates to document how AI tools were used. The final README should contain a transparent disclosure that distinguishes generated suggestions from engineering decisions and verification.

### Suggested disclosure content

- List the AI tools used (for example, Cursor AI and ChatGPT).
- Describe where AI was used: scaffolding, code suggestions, debugging, test ideas, documentation, UI iteration.
- State that generated code was reviewed and modified where needed.
- Explain the verification process: unit tests, API tests, manual offline/retry tests, and code review of synchronization logic.
- Identify important suggestions that were rejected or changed, especially around synchronization, duplicate prevention, conflicts, and data safety.
- State clearly that the submitted architecture and final implementation remain the developer's responsibility.

## Appendix D - References

The following external references informed terminology and the architecture direction. Product requirements themselves are based primarily on the supplied technical exercise specification.

- Next.js Documentation - overview of Next.js as a React framework for full-stack web applications and App Router concepts. [Ref: Next.js Documentation]
- Next.js Learn - App Router and full-stack course, including project structure and deployment concepts. [Ref: Next.js Learn - App Router]
- Product naming note፡የትም (Yetim) means "anywhere" in Amharic and is the basis of the brand story. [Product naming information supplied for this SRS]

### Final implementation note

This SRS is intentionally detailed enough to act as both a build contract and a reviewer-facing explanation of the engineering decisions behind Yetim. The highest-risk behavior is synchronization; implementation and testing should therefore treat the offline store, outbox, idempotency, versioning, and history model as first-class domain infrastructure rather than utility code.